

# Bulk schedule

Document #: P2209R0  
Date: 2020-08-14  
Project: Programming Language C++  
SG1  
Reply-to: Lee Howes  
<[lwh@fb.com](mailto:lwh@fb.com)>  
Lewis Baker  
<[lbaker@fb.com](mailto:lbaker@fb.com)>  
Kirk Shoop  
<[kirkshoop@fb.com](mailto:kirkshoop@fb.com)>  
Eric Niebler  
<[eniebler@fb.com](mailto:eniebler@fb.com)>

## § 1 Introduction

The design of bulk execution for executors has continued to evolve. In [P2181] Jared Hoberock and Michael Garland have proposed the `bulk_schedule` operation. This approximates an idea that had been circulating around SG1 for a while, in discussions with Daisy Hollman and others. In the form in the paper it falls a little short on a couple of fronts, but is moving in the right direction we hope to satisfy all parties. This paper serves as an addendum to that design that we hope leads to wider agreement.

### § 1.1 Properties of Sender/Receiver

The sender/receiver work, in its most general discussed form in the group, aims for a few goals:

- Lazy by assumption: to allow implementations to remove shared state and synchronization.
- Inline allocated where possible: connect/start APIs so that all asynchronous work can be inlined into the caller's state if that makes sense, rather than assuming heap allocations.
- Sequenced: `set_value` calls represent sequencing between asynchronous operations.
- Optimisable: `set_value` calls are not necessary if two chained operations can customise on each other's types.
- Composable: rather than building out variants of an algorithm (post/defer, allocator parameter, different function signatures, per-algorithm shared state, timeouts)

composition allows us to build more sophisticated algorithms from simple ones. With inline allocation and laziness we can do this at no cost.

- Interoperable: by defining first the hooks between a sender and a receiver we define interfaces between different implementors' libraries in the same way that a range defines a simple shared concept, irrespective of how an algorithm is implemented. This includes generic support for error handling, value propagation and cancellation.
- Cancellable: supporting cancellation in both bulk and non-bulk algorithms is important to a lot of use cases. In [P0443R13] the downstream part of cancellation is explicit in the inclusion of `set_done`. The upstream is a query on the receiver.

`bulk_schedule`, compared with `bulk_execute` gives us most of these features. This is its power. As a trivial example, if we want a blocking operation, we compose `bulk_schedule` with `sync_wait`:

```
sync_wait(bulk_join(bulk_transform(bulk_schedule(ex, 10), []()
{...})));
```

this requires the `bulk_transform` algorithm, but that is a trivial pass through much like [P1897]'s `transform` and `bulk_join` converts a bulk operation into a single operation.

If we want to wait with a timeout here, we can add that algorithm:

```
timed_sync_wait(bulk_join(bulk_transform(bulk_schedule(ex, 10), []
(){...})), 10s);
```

while maintaining a clearly written clue to what we are doing and where the caller will block.

The gap in the [P2181] definition of `bulk_schedule` is in sequencing of operations. This is what we aim to close.

## § 1.2 Proposal TL/DR

Make `bulk_schedule` symmetric with `schedule` by:

- Adding a `set_next` operation alongside `set_value`. This roughly matches [P2181]'s definition of `set_value` without changing the semantic of the operation.
- Define `many_sender` and `many_receiver` to match these definitions.
- Propagating the forward progress requirements of `set_next` via a `get_execution_policy` query on the `many_receiver`, consistent with `get_scheduler` as defined in [P1898].
- Propagating a `stop_token` via a `get_stop_token` query on the `many_receiver`, as defined in [P2175] and allowing the `many_sender` to abort early on cancellation.

## § 2 Sequencing

A sequence of executes is clearly a valuable thing. Let's assume here that `func2` depends on `func1`. We want to support chaining of operations in some fashion:

```
bulk_execution of func on exec
bulk_execution of func2 on exec
```

There are different ways we can enforce this ordering.

## § 2.1 Sequencing using blocking

One way is to make these blocking, either directly, or by manually using blocking algorithms to make it explicit at the call site:

```
sync_wait(bulk_execute(exec, func));
sync_wait(bulk_execute(exec, func2));
```

Blocking has a huge disadvantage because it is viral. If we have some algorithm that calls these two:

```
void bulk_algorithm(...) {
    sync_wait(bulk_execute(executor, func));
    sync_wait(bulk_execute(executor, func2));
}
```

then `bulk_algorithm` has to be blocking. For some algorithms this may be a valid design, but for asynchronous algorithms it is not, or at least not without launching a thread for each algorithm to do the blocking, so in general this is a poor solution. Blocking the caller would, after all, make the algorithm synchronous. We need true asynchronous algorithms, and so this cannot be the only way we provide work chaining.

## § 2.2 Sequencing using an implicit queue

Another approach is that we implicitly sequence, such that executor maintains an in-order queue in order of calls to `bulk_execute`. This is a common approach in runtime systems. The problem with it is that it does not interoperate, so it fails the *Interoperatable* goal. We need to use a second mechanism to bridge queues from different implementations.

That is to say that in this code, the question remains open:

```
void bulk_algorithm(...) {
    bulk_execute(facebook_executor, func);
    bulk_execute(nvidia, func2);
}
```

## § 2.3 Sequencing by nesting

We can sequence by nesting, where the implementation of `func` triggers the enqueue of `func2`. For scalar work this makes a lot of sense:

```

    executor.execute([executor]() {
        func();
        executor.execute(func2);
    });

```

It is a standard implementation strategy for Futures libraries, where the dependence graph is built outside of the executor library. The actual enqueue does not happen until the previous task completes. The implementation has no way to optimise this into an in-order queue, but for practical CPU libraries this generally works ok. It is how folly's Futures are implemented, and it is how coroutines generally work.

For a bulk algorithm this is more complicated. One way is that we maintain some mechanism to decide if we are in the last task to complete, maybe by incrementing an atomic:

```

    executor.bulk_execute([executor]() {
        func();
        if(is_last_task()) {
            executor.bulk_execute(func2);
        }
    });

```

We also need to be sure we can enqueue more work from within the bulk task. That may be limited for weak forward progress situations - it has traditionally not been possible on GPUs, for example and is likely to disallow vectorisation in other cases, which would be unfortunate.

## § 2.4 Sequencing using synchronization primitives

We could instead explicitly order on a barrier.

Either by having the caller block:

```

    some_barrier_type barrier(num_elements);
    bulk_execute(executor.schedule(), []() {
        func();
        barrier.signal();
    });
    barrier.wait();
    bulk_execute(executor.schedule(), []() {
        func();
    });

```

But this is equivalent to blocking, with the same problems.

Or by blocking within the task:

```

    some_barrier_type barrier(num_elements);
    bulk_execute(executor.schedule(), []() {
        func();
        barrier.signal();
    });

```

```
bulk_execute(executor.schedule(), [](){
    barrier.wait();
    func();
});
```

this has lifetime issues to consider with respect to barrier, fairly easily solved by reference counting and heap allocation. It also risks launching a wide fan out, blocking task, onto an execution context. This is an easy path to deadlock and so this is equivalent to blocking, in practice. Note that launching a scalar task in the middle is roughly the same, just moving the blocking elsewhere.

## § 2.5 Using senders

Finally, we can chain bulk algorithms the same way we chain scalar algorithms. This is the design we discussed in Prague for [\[P0443R13\]](#) where `bulk_execute` takes and returns a `Sender`:

```
auto s1 = bulk_execute(executor.schedule(), func);
auto s2 = bulk_execute(s1, func2);
```

In this design the work has a well-defined generic underlying mechanism for signalling, using a `set_value` call when all instances of `func` complete and when `func2` should run. As this `set_value` call is well-defined as an interface, we can mix and match algorithms and mix and match authors without problems.

The way this differs from the nesting design is that it is up to executor to decide how `set_value` is called:

- it may be fine to use the atomic-count last-worker-calls model.
- the runtime may wish to attach a callback to the work that is run on some runtime-controlled thread.
- if we have an underlying in-order queue or event DAG representation, of the sort that CUDA or OpenCL supports, we actually enqueue two tasks: one bulk task representing the set of `set_next` calls and one to call `set_value` to notify completion, potentially with a synchronization barrier in between if the model requires it.

The executor is the right place to make that decision. Making the decision anywhere else would be a pessimisation in generic code.

As for the scalar case, we can also optimise away the `set_value` call when we know all these types and customise the algorithm, using the sequential queue underneath. For example, an OpenCL runtime might use events or an in-order queue to chain work on the same executor, and then use a host queue or an event callback to transition onto some other executor completely safely. Making this up to the implementation, rather than up to the user to inject code into the passed function, offers scope for more efficient implementations.

## § 2.5.1 Summary

The point here is that **sequence points matter** to bulk algorithms. Most importantly, completion sequence points matter. We can easily handle the start of a bulk algorithm by delaying enqueue, at some cost if we would have preferred to rely on FIFO queuing. It is much harder to notify the completion of a parallel operation without executor support. Completion is by far the more important sequence point to include, and both can be optimised away by overloading in the library.

By encoding sequence points in the abstraction we put them under the control of the execution context. By default, because one of our goals is that this code be *Interoperable* of course this uses `set_value`, `set_done` or `set_error`; that's the interface we have agreed for senders and receivers. In practice, though, we can customise on the intermediate sender types and avoid that cost. So a default of well-defined sequencing with optimisation is more practical as a model than no sequencing, custom sequencing for each executor type or, maybe worst of all, blocking algorithms.

## § 3 The changes

To maintain sequencing, we need `set_value` to have the same definition as in sender for the type returned by `bulk_schedule`. It should not be possible to accidentally chain scalar work onto bulk work.

The list of clarifications necessary:

- `set_value` will, like `set_error` or `set_done` be called at most once on the receiver.
- We add a `set_next` operation, that differentiates a `many_sender` or `many_receiver` from their original forms, that is called once for each element of the iteration space, and passes the index.
- After all necessary calls to `set_next` return, `set_value` will be called.
- `set_error` or `set_done` indicate that some `set_next` calls may not have completed successfully.
- A query, `get_execution_policy` called on a `many_receiver` will return the policy that that receiver requires to be able to safely call `set_next` on the receiver. This may or may not be utilised by the scheduler, but it should be propagated through a chain of receivers and upgraded to a tighter policy if necessary.
- A query, `get_stop_token` called on a `many_receiver` will return a `stop_token` associated with that receiver such that the receiver or some downstream receiver may request cancellation of the operation.
- A `many_receiver` is-a receiver and when passed to a single sender behaves as if an iteration space of 0 was executed.
- A sender is-a `many_sender` by the same interpretation. It is a `many_sender` that does not call `set_next`.

- `bulk_schedule(s, size)` returns a `many_sender` that once connected to a `many_receiver r` will call `set_next(r, idx)` once for each in the range  $0 \leq \text{idx} < \text{size}$  and subsequently call `set_value(r)`.
- A `many_sender` **may** obtain a `stop_token` via a call to `get_stop_token` on a `many_receiver r`. If stop is requested the `many_sender` **may** elide subsequent calls to `set_next` and call `set_done(r)`.

## § 4 Impact on the Standard

### § 4.1 `execution::set_next`

The name `execution::set_next` denotes a customization point object. The expression `execution::set_next(R, Idx, Vs...)` for some subexpressions `R`, `Idx` and `Vs...` is expression-equivalent to:

`R.set_next(Idx, Vs...)`, if that expression is valid. If the function selected does not send the value(s) `Idx` and `Vs...` to the `many_receiver R`'s next operation, the program is ill-formed with no diagnostic required.

Otherwise, `set_next(R, Idx, Vs...)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void set_next();
```

and that does not include a declaration of `execution::set_next`. If the function selected by overload resolution does not send the value(s) `Idx` and `Vs...` to the receiver `R`'s next channel, the program is ill-formed with no diagnostic required.

Otherwise, `execution::set_next(R, Idx, Vs...)` is ill-formed.

[Editorial note: We should probably define what “send the value(s) `Vs...` to the `many_receiver R`'s next channel” means more carefully. —end editorial note]

### § 4.2 `execution::bulk_schedule`

The name `execution::bulk_schedule` denotes a customization point object. For some subexpressions `s` and `size`, let `S` be a type such that `decltype((s))` is `S` and `Size` be a type such that `decltype((size))` is `Size`. The expression `execution::bulk_schedule(s, size)` is expression-equivalent to:

`s.bulk_schedule(size)`, if that expression is valid and its type models sender.

Otherwise, `schedule(s)`, if that expression is valid and its type models sender with overload resolution performed in a context that includes the declaration

```
void bulk_schedule();
```

and that does not include a declaration of `execution::bulk_schedule`.

Otherwise, `execution::bulk_schedule(s)` is ill-formed.

[NOTE: The definition of the default implementation of `bulk_schedule` is open to discussion]

## § 4.3 Concept `many_receiver_of`

A `many_receiver` represents the continuation of an asynchronous operation formed from a potentially unordered sequence of indexed suboperations. An asynchronous operation may complete with a (possibly empty) set of values, an error, or it may be cancelled. A `many_receiver` has one operation corresponding to one of the set of indexed suboperations: `set_next`. Like a receiver, a `many_receiver` has three principal operations corresponding to the three ways an asynchronous operation may complete: `set_value`, `set_error`, and `set_done`. These are collectively known as a `many_receiver`'s completion-signal operations.

```
template<class T, class Idx, class... An, class... Nn...>
concept many_receiver_of =
    receiver_of<T, An...> &&
    requires(remove_cvref_t<T>&& t, Idx idx, An&&... an, Nn&&...
nn) {
    execution::set_value(std::move(t), (An&&) an...);
    execution::set_next(t&, idx, (Nn&&) nn...);
};
```

The `many_receiver`'s completion-signal operations have semantic requirements that are collectively known as the `many_receiver` contract, described below:

None of a `many_receiver`'s completion-signal operations shall be invoked before `execution::start` has been called on the operation state object that was returned by `execution::connect` to connect that `many_receiver` to a `many_sender`.

Once `execution::start` has been called on the operation state object, `set_next` shall be called for each index in some iteration space and parameterised with that index, under the restriction of the policy returned by a call to `get_execution_policy` on the `many_receiver`.

All of the `many_receiver`'s `set_next` calls shall happen-before any of the `many_receiver`'s completion-signal operations. Exactly one of the receiver's completion-signal operations shall complete non-exceptionally before the receiver is destroyed.

If any call to `execution::set_next` or `execution::set_value` exits with an exception, it is still valid to call `execution::set_error` or `execution::set_done` on the `many_receiver`. If all calls to `execution::set_next` complete successfully, it is valid to call `execution::set_value` on the `many_receiver`.



Once one of a many\_receiver's completion-signal operations has completed non-exceptionally, the many\_receiver contract has been satisfied.

## § 4.4 Concepts many\_sender, many\_sender\_to and typed\_many\_sender

XXX TODO The many\_sender and many\_sender\_to concepts. If these are necessary, complete definitions along with sender, sender\_to and typed\_sender.

```
template<class S>
    concept many_sender = ...

template<class S, class R>
    concept many_sender_to = ...

template<class S>
    concept typed_many_sender =
        many_sender<S> &&
        has-sender-types<sender_traits<remove_cvref_t<S>>>;
```

None of these operations shall introduce data races as a result of concurrent invocations of those functions from different threads.

A many\_sender type's destructor shall not block pending completion of the submitted receiver. [Note: The ability to wait for completion of submitted function objects may be provided by the associated execution context. –end note]

## § 4.5 Extend sender\_traits

Let has-sender-types be an implementation-defined concept equivalent to:

```
...
template<template<template<class...> class> class>
    struct has-next-types ; // exposition only

...

template<class S>
    concept has-many-sender-types =
        requires {
            typename has-value-types <S::template value_types>;
            typename has-next-types <S::template next_types>;
            typename has-error-types <S::template error_types>;
            typename bool_constant<S::sends_done>;
        };
```

If has-many-sender-types<S> is true, then many-sender-traits-base is equivalent to:

```
template<class S>
    struct sender_traits-base {
        template<template<class...> class Tuple, template<class...>
class Variant>
```

```

        using value_types = typename S::template
value_types<Tuple, Variant>;

        template<template<class...> class Tuple, template<class...>
class Variant>
            using next_types = typename S::template next_types<Tuple,
Variant>;

        template<template<class...> class Variant>
            using error_types = typename S::template
error_types<Variant>;

        static constexpr bool sends_done = S::sends_done;
    };

```

Otherwise, let `void-many-receiver` be an implementation-defined class type equivalent to

```

struct void-many-receiver { // exposition only
    void set_value() noexcept;
    void set_next(size_t) noexcept;
    void set_error(exception_ptr) noexcept;
    void set_done() noexcept;
};

```

## § 5 Why use `get_execution_policy` on the receiver?

There are two questions embedded in this:

- Why pass the policy into the bulk operation at all (with reference to the discussion point in [P2181])?
- Why do it this way rather than as a parameter?

### § 5.1 Why pass a policy into the bulk API?

For a compound algorithm, it is unreasonable to assume that the policy passed to the algorithm need be the one applied to the executor. This is for a variety of reasons, but primarily that the author of the algorithm may not be in a position to match policies perfectly. We might implement sort:

```

output_sender std::execution::sort(input_sender, executor, policy,
comparison_function, range);

```

where the policy passed to match `comparison_function` is `par_unseq` but where a complex multi-stage sort needs `par` internally. Here we end up with a question: do we have to restrict the interface to match the implementation? That might be hard. The alternative is to transform the executor inside the algorithm. Potentially using `require`, or a `with_policy` wrapper.

The problem is that modifying the executor to provide a policy is semantically identical to passing it with the continuation, but it is less easy to read:

```
output_sender std::execution::algorithm(input_sender, executor,
policy) {
    auto seq_executor = with_policy(executor, seq);
    auto s1 = alg_stage_1(input_sender, seq_executor)

    auto par_executor = with_policy(executor, par);
    return alg_stage_2(s1, par_executor)
}
```

it is harder to read because the executor has been transformed and stored - it may drift far from the point of use, and thus create a risk of UB introduced during maintenance.

The alternative would appear structurally similar, but note that if we have attached a policy to the algorithm *if the executor is incapable of executing that way it can report the error*. Whichever way we do this we have to decide what an executor is allowed to reject in terms of itself being associated with a policy, and the work being associated with a policy.

## § 5.2 Why use `get_execution_policy` on the receiver instead of a parameter?

This is a question both of scaling of compound algorithms and into the future. A compound algorithm may need to make policy decisions at each stage. For example, using async versions of `std::sort` and `std::transform`

```
output_sender std::execution::algorithm(input_sender, executor,
policy) {
    auto s1 = sort(input_sender, executor, policy)
    return transform(s1, executor, policy)
}
```

based on prior work, each of these user-visible algorithms would take a policy. They are all user-facing as well as implementation-details. In the `get_execution_policy` model each of these can communicate a policy to the prior algorithm.

We do realise that execution policy is a broader concept than just forward progress - it may be that we need to separate concerns here. The aspect of the execution policy that is a statement of what the most relaxed conditions in which the function is safe to call the way they are used in the current set of defined policies on the parallel algorithms, may be abstracted in this way while the wider policy is separate and describing execution may be useful. In that case `get_forward_progress_requirement` might be a better name.

Having said that, wider policy requirements still do not seem to be properties of the executor and so the above section about passing a property into the bulk API still apply. We should be careful to distinguish properties of an individual algorithm call: which may include the forward progress requirements of the operation, the allocator the operation

wants to use, the cancellation token it wants to be able to cancel with, if it is following a continuation or run-anytime model of execution or even the executor it wants to have work run on (See [P1898]). These are distinct from properties that should be applied to an executor, such as how it is capable of running things, potentially also an allocator that it wants to use, and similar work.

There is also an argument for needing a separate forward progress query representing the `set_value` call, in addition to the bulk `set_next` calls. Both propagating through a receiver chain via `tag_invoke` forwarding. If `set_value` is to be called on the last completing task, and we know that the next algorithm constructed a receiver that is safe to call in a `par_unseq` agent, then the prior agent is safe to call it from its last completing `par_unseq` agent. If not, then the executor has to setup a `par` agent to make that call from, because that is the most general method for chaining work across different contexts. However, this is a bigger problem to solve. If we want to implement nested parallelism, or to call any parallel algorithm, synchronous or asynchronous, from within a parallel algorithm then we need to understand the forward progress requirements that call places. For that matter, if we call any uncontrolled code from within a parallel algorithm, the same applies.

Another future use case reason for doing it this way is the potential for compiler help. Imagine something like:

```
bulk_schedule(exec, size, bulk_transform([]()
[[with_compiler_generated_policy]] {...}))
```

where the compiler is allowed to analyse that code and wrap the lambda in a way that will expose the `get_execution_policy` query to `bulk_transform` and of course to `bulk_schedule`. If the compiler sees a `std::mutex` in here, it might strengthen the policy requirement such that executor can report a mismatch. We have provided a general mechanism, that future compilers can hook into, especially when generating custom accelerator code, to reduce the risk of mistakes on the part of the developer.

## § 6 References

[P0443R13] 2020. A Unified Executors Proposal for C++.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0443R13.html>

[P1897] 2020. Towards C++23 executors: A proposal for an initial set of algorithms.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1897r3.pdf>

[P1898] 2020. Forward progress delegation for executors.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1898r1.pdf>

[P2175] 2020. Composable cancellation for sender-based async operations.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2175r0.pdf>

[P2181] 2020. Correcting the Design of Bulk Execution.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2181r0.pdf>